

Tema 8.4: CSS Moderno y Avanzado — Custom Properties, Nesting y Container Queries

Resumen

En esta lectura, usted estudiará las capacidades más avanzadas y modernas incorporadas en los estándares de **CSS (Cascading Style Sheets)**. Aprenderá a declarar y manipular **CSS Custom Properties** (variables de CSS), a utilizar el **Nesting nativo** (anidamiento de reglas) para escribir código limpio, a diseñar layouts ultrarresponsivos con **CSS Container Queries** y a aplicar técnicas avanzadas de **CSS Grid** como `minmax()` y `auto-fit` sin requerir hojas de estilo imperativas de terceros.

Concepto Principal (El 'Qué')

La evolución reciente de CSS ha introducido características nativas potentes que antes dependían de preprocesadores (como Sass o Less) o frameworks pesados. Estas herramientas mejoran la mantenibilidad de la interfaz de usuario y permiten un diseño responsivo granular a nivel de componente.

Las tecnologías clave a estudiar son:

CSS Custom Properties (Variables de CSS): Permiten almacenar valores reutilizables (colores, fuentes, medidas) en cascada. Se declaran prefijándolas con dos guiones (`--nombre-variable`) y se consumen con la función `var(--nombre-variable)`. A diferencia de las variables de Sass, las Custom Properties son **dinámicas**, se evalúan en tiempo de ejecución en el navegador y pueden ser modificadas mediante JavaScript o redefinidas dentro de Media Queries.

Nesting Nativo (Anidamiento): Permite agrupar selectores de CSS anidándolos unos dentro de otros, reflejando de forma directa la estructura jerárquica del marcado HTML. Esto reduce significativamente la duplicación de selectores y la redundancia del código fuente. Se

utiliza el símbolo & para hacer referencia explícita al selector padre inmediato.

Container Queries (Consultas de Contenedor): Las Media Queries tradicionales responden al ancho de la pantalla completa (*viewport*). Sin embargo, un componente (como una tarjeta) puede posicionarse tanto en una barra lateral estrecha como en el área principal de contenido ancho. Las **Container Queries** (`@container`) permiten evaluar y aplicar estilos en función de las dimensiones físicas de su **contenedor directo** (el *container host*), independientemente del tamaño de la pantalla global.

CSS Grid Moderno (minmax + auto-fit): Permite diseñar distribuciones responsivas elásticas sin utilizar Media Queries de tamaño fijo. Al combinar `grid-template-columns: repeat(auto-fit, minmax(250px, 1fr))`, el navegador calcula y ajusta automáticamente la cantidad de columnas en base al espacio disponible, distribuyendo el excedente de forma proporcional.

Diferencia entre Media Queries y Container Queries:

[Pantalla Grande (Viewport)]

■■■ [Sidebar Estrecho (300px)] <-- Container Query aplica estilos para espacio estrecho

■■■ [Contenedor Principal (800px)] <-- Container Query aplica estilos para espacio ancho

Analogía (La Intuición)

Considere el CSS avanzado como los **materiales y planos de construcción de un mueble modular moderno**:

- * **Las Custom Properties son las latas de pintura centralizadas:** En lugar de ir a pintar cada cajón y tornillo de azul de forma manual, usted crea una manguera de alimentación principal para la pintura azul (`--color-mueble: blue;`). Si el cliente decide cambiar a verde, usted solo reemplaza la lata en el suministro central y todo el mueble se repinta automáticamente.
- * **El Nesting nativo es la organización jerárquica de las instrucciones:** Es el manual estructurado que dice: *"Para la cajonera, haga X; y dentro de la cajonera, para los tiradores, haga Y"*, en lugar de dar instrucciones repetitivas y aisladas para cada pieza pequeña.
- * **Las Container Queries son el diseño elástico e inteligente de los cajones:** Si un cajón modular se coloca en un mueble pequeño de oficina (espacio estrecho), sus separadores se apilan verticalmente; si el mismo cajón se inserta en un armario de sala grande (espacio ancho), los separadores se expanden horizontalmente por sí mismos. El cajón sabe cómo comportarse según el espacio de la gaveta que lo contiene, no de la casa completa.

Ejemplo de Código e Implementación (El 'Cómo')

A continuación se detalla la implementación de una cuadrícula adaptable que utiliza variables, anidamiento nativo y Container Queries para alterar el diseño visual de los componentes en base a su espacio disponible:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Demostración de CSS Moderno</title>
  <style>
    /* 1. Declaración de variables en el ámbito raíz (global) */
    :root {
      --color-primario: #2563eb;
      --color-fondo-tarjeta: #ffffff;
      --color-texto: #1e293b;
      --color-borde: #e2e8f0;
      --padding-tarjeta: 20px;
      --radio-borde: 8px;
    }

    body {
      background-color: #f8fafc;
      color: var(--color-texto);
      font-family: system-ui, -apple-system, sans-serif;
      padding: 40px;
      margin: 0;
    }

    /* 2. Cuadrícula de diseño flexible elástica sin Media Queries */
    .cuadrícula-adaptable {
      display: grid;
      grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
      gap: 20px;
      margin-bottom: 40px;
    }

    /* 3. Definición del Contenedor Padre para Container Queries */
    .contenedor-tarjeta {
      /* Define que este elemento será monitoreado en su dimensión de ancho (inline-size) */
      container-type: inline-size;
      width: 100%;
    }
  </style>
</head>
<body>
  <div class="cuadrícula-adaptable">
    <div class="contenedor-tarjeta">
      <div class="tarjeta">
        <h2>¡Hola Mundo!</h2>
        <p>Este es un ejemplo de CSS Moderno utilizando variables, container queries y CSS Grid. ¡Espero que te guste!</p>
      </div>
    </div>
  </div>
</body>
</html>
```

```

    }

    /* 4. Implementación del componente utilizando Nesting Nativo */
    .tarjeta-info {
        background-color: var(--color-fondo-tarjeta);
        border: 1px solid var(--color-borde);
        border-radius: var(--radio-borde);
        padding: var(--padding-tarjeta);
        box-shadow: 0 1px 3px rgba(0,0,0,0.1);
        display: flex;
        flex-direction: column;
        gap: 12px;

        /* Anidamiento de título */
        h3 {
            margin: 0;
            color: var(--color-primario);
            font-size: 1.25rem;
        }

        /* Anidamiento de párrafo */
        p {
            margin: 0;
            color: #64748b;
            line-height: 1.5;
        }

        /* Anidamiento y modificación de estado en hover */
        &:hover {
            border-color: var(--color-primario);
            box-shadow: 0 4px 6px -1px rgba(37, 99, 235, 0.1);
        }
    }

    /* 5. Container Query: Aplica estilos si el CONTENEDOR directo mide más de 500px */
    @container (min-width: 500px) {
        .tarjeta-info {
            /* Cambiar la dirección de la tarjeta a fila (horizontal) */
            flex-direction: row;
            align-items: center;
            justify-content: space-between;

            h3 {
                font-size: 1.5rem;
                flex: 1;
            }
        }
    }

```

```

    p {
        flex: 2;
        padding-left: 20px;
    }
}
}
</style>
</head>
<body>

<h1>Layout Adaptable con CSS Moderno</h1>

<!-- Contenedor en cuadrícula estándar -->
<div class="cuadrícula-adaptable">

    <!-- Elemento 1: Su contenedor mide < 500px, se renderizará vertical -->
    <div class="contenedor-tarjeta">
        <div class="tarjeta-info">
            <h3>Componente Estrecho</h3>
            <p>Este componente se muestra vertical porque su contenedor inmediato tiene dimensio
        </div>
    </div>

    <!-- Elemento 2: Su contenedor mide < 500px, se renderizará vertical -->
    <div class="contenedor-tarjeta">
        <div class="tarjeta-info">
            <h3>Otro Componente</h3>
            <p>Ideal para rejillas de tarjetas dinámicas, adaptándose automáticamente al espacio
        </div>
    </div>

</div>

<!-- Contenedor Ancho Solitario: Ocupa el 100% de la pantalla (generalmente > 500px) -->
<div class="contenedor-tarjeta">
    <div class="tarjeta-info">
        <h3>Componente Horizontal</h3>
        <p>Este componente se adapta horizontalmente de forma automática porque su contenedor di
    </div>
</div>

</body>
</html>

```

Explicación Línea por Línea

- **container-type: inline-size:** Indica al motor del navegador que monitoree constantemente el ancho de este elemento. Cualquier hijo de este nodo puede hacer consultas de tamaño sobre él.
- **@container (min-width: 500px):** Evalúa el contenedor marcado con `container-type`. Si este mide 500px o más, aplica las reglas internas a sus descendientes.
- **repeat(auto-fit, minmax(280px, 1fr)):** Le indica a la cuadrícula que repita tantas columnas como quepan (`auto-fit`), asegurando que ninguna columna mida menos de 280px ni más de la fracción equitativa disponible (`1fr`).
- **&:hover:** El selector anidado mediante `&` se traduce como `.tarjeta-info:hover`, lo cual mantiene todos los estilos del bloque organizados jerárquicamente dentro de una sola declaración CSS principal.

Trampas Comunes (Gotchas)

1. **Olvidar Definir el Contenedor (Container Host):** Si usted escribe una regla `@container` pero olvida aplicar la propiedad `container-type` (o `container`) al elemento padre correspondiente, la consulta de contenedor fallará silenciosamente y los estilos no se aplicarán nunca. El navegador no adivina qué elemento es el contenedor de referencia.

Solución: Verifique siempre que el padre directo (o algún ancestro) tenga declarado `container-type: inline-size` o `container-type: size`.

Diferencias de Sintaxis de Variables CSS en Mayúsculas/Minúsculas: A diferencia de las propiedades comunes de CSS, las Custom Properties **son estrictamente sensibles a mayúsculas y minúsculas** (*case-sensitive*). Si usted define `--Color-Primario` e intenta consumirlo como `var(--color-primario)`, el navegador ignorará la variable silenciosamente y utilizará el valor de respaldo o la propiedad por defecto.

4. *Solución:* Siga una convención de nomenclatura estándar en minúsculas y utilizando guiones medios (*kebab-case*).

Pregunta de Reflexión

Si usted define una variable CSS dentro del selector `:root` y luego declara la misma variable con otro valor dentro del selector de clase `.tarjeta-info`, explique qué valor tomará un elemento hijo que consuma dicha variable y detalle cómo el concepto de la **cascada CSS** influye en esta resolución de estilos en tiempo de ejecución.